大模型训练框架与基础设施 横纵分析报告

> **研究对象**：大模型训练框架与基础设施（分布式并行训练、显存优化、计算加速）

> **类型**：技术范式

> **研究日期**：2026-04-30

一、一句话定义

大模型训练框架与基础设施，是支撑千亿乃至万亿参数模型在 GPU 集群上高效训练的软件系统与硬件架构的总和。它解决的核心矛盾是：**模型的增长速度（每年约 10×）远远超过了单卡算力和显存的增长速度（每年约 1.5×）**，必须通过分布式并行技术把成百上千张 GPU 组合成一台"虚拟超级计算机"。

从用户视角看，训练框架是一套"多卡协作的操作系统"——它帮用户管理三件事：**调度**（什么卡算什么）、**通信**（卡之间怎么传数据）、**存储**（参数和中间结果放哪里），让用户只需写模型代码。框架的使命是：用 N 张卡的算力之和，逼近 N 倍的训练速度。

二、技术背景

> 前置知识已独立为 `大模型训练框架与基础设施_前置知识.md` ，零基础读者请先阅读。本节省略基础概念，仅做快速串联，让读者在进入纵向叙事前有框架感。

2.1 核心矛盾：模型的膨胀 vs 硬件的蜗行

这个领域存在的根本原因可以用一个不等式概括：

模型参数增速（~10×/年） >> 单卡显存增速（~1.5×/年）

GPT-1（2018）是 117M 参数，GPT-3（2020）是 175B 参数——两年 1,500×。而同期 GPU 显存从 V100 的 32GB 增长到 H100 的 80GB——only 2.5×。

这意味着：你不用分布式训练，根本别想训练一个现代大模型。

2.2 解决方案的三层结构

并行策略层面：数据并行（DP）→ 参数分片（ZeRO/FSDP）→ 张量并行（TP）→ 流水线并行（PP）→ 序列并行（SP）→ 专家并行（EP）。每一层解决一个不同的"放不下"问题：模型太大（TP/PP）、显存太浪费（ZeRO）、序列太长（SP）、Expert 太多（EP）。

计算优化层面：FlashAttention 1→2→3 通过 IO-aware 算法将 Attention 的显存带宽需求从 $O(N^2)$ 降到 $O(N)$。混合精度训练从 FP32 到 FP16/BF16 到 FP8，精度每降一级，吞吐量几乎翻倍。

硬件架构层面：NVLink 提供节点内 600-900GB/s 的带宽，而跨节点的 InfiniBand 只有 50GB/s——差了 10 倍以上。这决定了通信密集的策略（TP）必须放在节点内部，通信量小的策略（PP、ZeRO）才能跨节点。**训练框架本质上是在这些硬件约束下做最优的资源分配。**

2.3 ZeRO 的颠覆性（重要）

ZeRO（Zero Redundancy Optimizer）是训练框架领域最重要的单项创新。它的核心洞察极简：传统数据并行中，N 张卡存了 N 份完全相同的优化器状态。每张卡上 Adam 需要为每个参数保存 momentum（fp32）和 variance（fp32）——两倍于参数本身的显存。N 张卡就是 N 份冗余。

ZeRO 的解决方案更简单：不存。把优化器状态、梯度、参数量逐级分片到所有卡上，用到时通过通信即时收集。Stage 1 分片优化器状态（节省 4×），Stage 2 分片梯度（节省 8×），Stage 3 分片参数（节省 N×）。

这个想法在今天看来如此自然，但在 2019 年它是反直觉的——"通信比显存贵"是当时的普遍认知。ZeRO 证明了：**当显存成为硬约束时，多花点通信换显存是划算的。**

三、纵向分析：从单卡训练到万卡集群

> 从 2014 年的 DataParallel 到 2026 年的 FP8 万卡训练，一段加速主义的历史。

3.1 前传（2014-2019）：数据并行的时代

2014-2015 年，深度学习模型还小。AlexNet 只有 60M 参数，VGG 是 138M——单张 GPU（K80 12GB、Titan X 12GB）轻松放下。多 GPU 训练只是为了"跑得快"，不是"能不能跑"。

2015 年 11 月，TensorFlow 开源。 它不仅带来了静态计算图和自动微分，还内置了 `tf.distribute`——第一个主流框架级的分布式训练方案。当时 Google 内部已经用 TensorFlow 训练大规模的序列模型（神经机器翻译），数据并行的基本模式已确立：每卡一份完整模型、各算各的梯度、AllReduce 同步更新。

TensorFlow 的分布式方案在工程上是可靠的，但开发者体验糟糕——静态图调试困难、`tf.distribute` 的抽象层晦涩难懂。这为 PyTorch 的崛起埋下了伏笔。

2016 年 9 月，PyTorch 开源。 它来自 Meta 的 FAIR 团队（Soumith Chintala、Adam Paszke 等），设计哲学是"Pythonic"——动态图、调试友好、跟 Python 生态无缝集成。但早期的 PyTorch 完全没有分布式训练支持。这一点在当时被很多人认为是"生产环境不可用"的证据。

直到 **2019 年 2 月，PyTorch 1.0 正式发布**，才带来了 `torch.nn.parallel.DistributedDataParallel`（DDP）。DDP 的设计跟早期的 `DataParallel` 完全不同：`DataParallel` 是单进程多线程，GIL 导致无法真正并行；DDP 是

多进程（每个 GPU 一个进程），通过 NCCL 做 AllReduce 通信，同时在反向传播时异步执行梯度同步，不阻塞计算。

DDP 的 API 极其简洁——只需用 `nn.parallel.DistributedDataParallel` 包裹模型——这让 PyTorch 在分布式训练上的易用性大幅超过了 TensorFlow。**2019 年是 PyTorch 从"研究工具"走向"生产框架"的关键转折年。**

2018–2019 年同时是模型规模的临界点。 BERT（340M）和 GPT-2（1.5B）让单卡训练变得不可能——V100 的 32GB 显存放不下超过 1B 参数的模型。数据并行遇到天花板：每张卡必须存完整模型，模型越大需要的 GPU 越多，而大数据并行在超过一定规模后，AllReduce 的通信开销开始吞噬扩展收益。

但 2019 年最大的新闻不是任何框架更新。它是两个后来重塑了整个行业的论文的提交年份——Megatron-LM 和 ZeRO。

3.2 双星闪耀（2019–2020）：Megatron-LM 和 DeepSpeed

2019 年 9 月 17 日，NVIDIA 的 Mohammad Shoeybi、Mostofa Patwary 和 Bryan Catanzaro 在 arXiv 上提交了 Megatron-LM 论文。 他们用量极小的代码改动，解决了一个被认为需要编译器或新框架才能解决的问题：Tensor Parallelism（TP）。

Megatron 的洞察是：Transformer 的多头注意力天然可并行。12 个头，分配到 12 张卡上各自计算，之后一次 AllReduce 组合结果。不是 Layer 之间的并行——那是 Pipeline Parallelism——而是**同一层内部的并行**。MLP 层的权重矩阵也按列和行两个方向切分：第一个线性变换按列切分（每卡算部分隐藏维度），第二个线性变换按行切分（AllReduce 后再组合）。

实验数据很有说服力：在 512 张 V100 上训练 8.3B 的 GPT-2，达到 76% 的扩展效率。耗时 21 分钟完成过去需要数天的任务。

但 NVIDIA 的开源风格是"先发论文，再缓缓开源"。Megatron 的代码直到 2021 年 3 月才公开——论文发表后 1.5 年。这个窗口期被微软抓住了。

2019 年 10 月 4 日——离 Megatron 论文仅差 17 天——微软研究院的 Samyam Rajbhandari、Jeff Rasley 和 Yuxiong He 提交了 ZeRO 论文。 这篇论文的标题很直白：《ZeRO: Memory Optimizations Toward Training Trillion Parameter Models》。

ZeRO 选了跟 Megatron 完全不同的切入点。Megatron 解决的是"模型太大"（切分计算），ZeRO 解决的是"显存太浪费"（消除冗余）。论文用一个简单的公式揭示了问题：训练一个 Ψ 参数的模型，用 Adam + mixed precision，每张卡需要 22Ψ bytes 的显存。对于 175B 模型，这是 3,850GB。其中只有极少量是真正"必须"的。

ZeRO 三阶段的演变是推理的层层递进：

- Stage 1（Optimizer State Partitioning）：Adam 的 momentum 和 variance 占了显存大头，但它们是可以分片的。每卡只更新自己那部分参数对应的优化器状态。
- Stage 2（Gradient Partitioning）：既然优化器状态已经分片了，梯度也可以直接 ReduceScatter——不再 AllReduce 后每卡存完整梯度，而是 ReduceScatter 后各存一份。
- Stage 3（Parameter Partitioning）：参数也分片。前向计算时从各卡 AllGather 参数，用完后丢弃非本卡的分片。通信量翻倍，但显存节省跟 GPU 数成线性关系。

2020 年是 DeepSpeed 拉开架势的一年。 2 月 10 日，DeepSpeed 正式开源，ZeRO Stage 1 可用，同时发布 Turing-NLG 17B 模型作为能力的展示。5 月，ZeRO-2（Stage 1+2）发布，支持 200B 参数训练，带来 10× 加速。9

月，ZeRO-Infinity 发布——Stage 3、ZeRO-Offload（卸载到 CPU/NVMe）、3D 并行（ZeRO + Megatron TP + PP）三位一体，支持万亿参数训练。

有一条时间线值得反复咀嚼：**2020 年 9 月时，DeepSpeed 已经具备了训练万亿参数模型的能力，而 GPT-3（175B，2020 年 7 月发布）才刚刚证明"大模型"这个方向是对的。** 训练框架的进步在模型需求之前先跑到了。这在整个 AI 史上是极其罕见的——通常情况下永远是模型需求倒逼工具进化。

ZeRO-Offload 值得一提。它解决的问题是"单卡显存还是不够怎么办"——把优化器状态和梯度卸到 CPU 内存，GPU 只做计算密集的前向和后向，优化的 Adam 更新在 CPU 上并行完成。ZeRO-Infinity 更进一步：把参数也卸到 NVMe SSD，在 GPU、CPU 内存、NVMe 三级存储之间做分层调度。

3.3 融合与分化（2021-2022）：Megatron-DeepSpeed 与 FSDP

2021 年 5 月，微软和 NVIDIA 宣布了一项重要合作：Megatron-DeepSpeed。 两套当时最顶尖的训练技术栈正式融合——DeepSpeed 提供 ZeRO 和 Pipeline Parallelism，Megatron 提供 Tensor Parallelism。两者互补（TP 管层内、PP 管层间、ZeRO 管显存），组成完整的 3D 并行方案。

合作的成果是线性的：在 800 张 V100 上以接近完美的扩展效率训练万亿参数模型。这可能是 AI 基础设施历史上最成功的一次开源合作——两家你死我活的竞争对手在技术上做了深度的整合。

合作还确立了一个范式至今不变的**拓扑映射规则**：

- TP 组内通信最密集（每层 AllReduce），放在节点内部（NVLink 600GB/s）
- PP 组间通信最小（只有层边界的一次传输），可以跨节点
- DP/ZeRO 组间通信中等（AllGather 参数），放在跨节点，利用 ZeRO++ 做通信压缩

2022 年 3 月，PyTorch 1.11 发布 FSDP（Fully Sharded Data Parallel）。 FSDP 本质上实现了 ZeRO Stage 3 的全部功能——参数、梯度、优化器状态全部分片——但它是 PyTorch 原生的。

为什么 Meta 选择自己做 FSDP 而不是直接用 DeepSpeed？这不是技术选择，是战略选择。**PyTorch 是 Meta 开源生态皇冠上的明珠，不能让分布式训练这个核心能力挂在微软的框架上。** FSDP 让 PyTorch 在"大模型训练"这个关键能力上实现了自给自足。

这个决策在社区层面造成了长期的格局分化: DeepSpeed 那边功能更全（Offload、MoE 支持、RLHF、通信压缩），FSDP 这边用起来更舒服（跟 PyTorch 生态紧密集成、torch.compile 兼容）。大厂内部大规模训练多用 DeepSpeed/Megatron，中小团队和 HuggingFace 生态更倾向 FSDP。

2021-2022 年还有两个值得关注的入场者。 2021 年 10 月，新加坡国立大学的尤洋（Yang You）团队提交了 ColossalAI 论文，后来成立了潞晨科技。ColossalAI 的主打差异点是"自动并行策略搜索"——用户只需要写单卡代码，框架自动搜索最优的 TP/PP/DP 组合。这个理念在学术上很吸引人，但在大规模训练实践中并未成为主流——专业用户不信任自动搜索的效率，更相信自己调出来的手动策略。

2022 年 1 月，UC Berkeley 的 Alpa 论文被 OSDI 2022 接收，也走了自动并行路线。跟 ColossalAI 同样的命运——被大厂实践冷落。**自动并行可能不是技术问题，而是信任问题：当你花几百万美元跑一次训练时，你不敢让一个算法替你决定并行策略。**

3.4 IO 觉醒（2022-2024）：FlashAttention 改写 Attention 的底层逻辑

如果说 ZeRO 改变了显存管理的范式，FlashAttention 改变了 Attention 计算的范式。它可能是 2020 年代以来**单项影响力最大**的训练优化技术。

2022 年 5 月，Stanford 博士生 Tri Dao 和 Christopher Re 团队提交了 FlashAttention 论文，被 NeurIPS 2022 接收。 论文的核心洞察极其根本：Attention 的瓶颈不是算力（FLOPS），而是显存带宽（memory bandwidth）。

标准 Attention 实现需要计算一个 N×N 的注意力分数矩阵，把它完整地写出到 HBM（高带宽显存），再从 HBM 读回来计算 softmax——两次 HBM 访问，每次 $O(N^2)$。当 N=4096 时，这个矩阵就是 16M 个元素；当 N=128K 时，它是 16B 个元素——已经超过了大部分 GPU 的 HBM 容量。

FlashAttention 的解决方案：通过 tiling（分块），在 GPU 的 SRAM（高速缓存）上完成 Attention 的子计算，只把最终结果写回 HBM。**HBM 读写从 $O(N^2)$ 降到 $O(N)$。** 同时通过不保存完整的 Attention 矩阵（反向时重新计算），进一步节省显存。

实验数据：BERT 训练提速 15%，GPT-2 训练提速 3×。对于一个"纯工程优化"而非"算法创新"的工作，这个提升是惊人的。

2023 年 7 月，FlashAttention 2 发布。 此时 Tri Dao 已经从 Stanford 毕业，加入了 Together AI。FA2 没有引入新的算法思想，而是做了更精细的工程优化：更好的线程块并行分配、减少非 matmul 操作、更高效的 causal masking。结果是比 FA1 再快约 2×，达到 A100 FP16/BF16 理论峰值的 50-73%。FA2 还部分使用了 OpenAI 的 Triton 编译器——这是 Triton 在生产级 kernel 中的一次重要应用。

2024 年 7 月，FlashAttention 3 发布。 这次是 NVIDIA 的 Jay Shah 和 Tri Dao 的联合成果。FA3 深度利用 Hopper H100 的新硬件特性——FP8 Tensor Core 和异步 WGMMA（Warp Group Matrix Multiply-Accumulate）指令——在 FP16 下达到了 740 TFLOPS，在 FP8 下达到了 1.2 PFLOPs。

FA1→2→3 的演进揭示了一个更深的规律：**AI 框架的创新正在从"纯算法创新"转向"硬件-软件协同设计"。** FA1 用通用 CUDA 在 Ampere 上实现，FA3 深度绑定 Hopper 的硬件特性。每一代新硬件都打开了新的优化空间，但代价是代码的可移植性降低——FA3 的核心优化在 A100 上无法使用。

3.5 进入深水区（2024-2026）：FP8、MoE、长上下文

2024 年，训练框架进入功能深化与整合阶段。

3 月：DeepSpeed-MoE 发布。 Mixtral 8×7B、DeepSeek-V2、GPT-4（据传）等 MoE 模型的兴起带来了新的并行需求——Expert Parallelism（EP）。MoE 的 router 把每个 token 分发给 Top-K 个 Expert，不同的 Expert 分布在不同的 GPU 上。这个"token 分发—计算—收集"的模式跟 TP/PP 完全不同，用的是 All-to-All 通信而非 AllReduce。DeepSpeed-MoE 在 ZeRO 的基础上增加了 EP 支持，在 MoE 训练场景下比纯 DP 方案节约 5-10× 的通信量。

7 月：Megatron Core 发布。 NVIDIA 宣布对 Megatron-LM 进行架构级重构——从一个大仓库式的单体库变为模块化架构。Megatron Core 支持 FP8 训练、多模态训练（文本+图像）、超长上下文。版本迭代很快：Core 0.16.0（2026 年 2 月），Core 0.17.0（2026 年 4 月）。

这个重构标志着 NVIDIA 开源策略的转变——从 2019 年的"先发论文、缓缓开源"到 2024 年的"快速迭代、社区驱动"。

12 月：DeepSeek-V3（671B MoE）发布。 它训练在 2,048 张 H800 上，使用了自研的 HAI-LLM 框架（基于 PyTorch）。DeepSeek-V3 最重要的贡献不是模型能力——而是**首次在大规模生产中全面验证了 FP8 训练的可行性**。MFU 达到约 50%+，证明了 FP8 训练不只是学术玩具。

DeepSeek-V3 还引入了 Multi-Token Prediction（MTP）——让模型同时预测多个后续 token。这需要训练框架支持特殊的 loss 计算和梯度传播。展示了框架层面需要为新的训练目标（不只是传统的 next-token prediction）提供灵活支持。

2025-2026 年，框架竞争进入"功能完备"阶段。 各家在基本功能上趋同——都支持 3D 并行、FlashAttention、FP8、MoE EP。差异化从"谁的功能多"变为"谁的体验好、谁的生态强"。DeepSpeed 靠微软生态，Megatron 靠 NVIDIA 硬件绑定，FSDP 靠 PyTorch 原生集成。

这个阶段的另一个变化是训练框架和推理框架的边界在模糊。vLLM、SGLang 等推理框架开始扩展训练能力（微调、对齐），而 DeepSpeed 和 Megatron 本身就包含推理模块。'训练-推理一体化'的趋势在加速——同一个框架处理从训练到部署的全生命周期。

两个新的方向在 2026 年加速：

长上下文训练（128K-1M token）：Ring Attention（基于 P2P 通信的环状序列并行）和 DeepSpeed Ulysses（基于 All-to-All 通信的序列并行）两种方案并行发展。两者各有优劣：Ulysses 适合同构集群，Ring Attention 更灵活但需要更多通信轮次。框架层面需要在这两者之间做兼容。

多模态训练：不同模态使用不同架构（文本=Transformer、图像=ViT、视频=3D Conv），需要框架支持异构并行——不同部分的 TP/PP 策略不同。

3.6 真实训练案例：从 GPT-3 到 DeepSeek-V3

理论和技术讲得再多，不如看几个真实案例来得直观。

GPT-3（175B，2020 年 7 月）：训练在微软 Azure 的 NDv4 集群上，使用了 10,000 张 V100 GPU。训练框架是微软内部实现的定制方案——不是公开的 DeepSpeed（当时 DeepSpeed 刚发布到 Stage 1，3D 并行还不成熟）。训练耗时约 14.8 天，总算力消耗约 3.14×10^{23} FLOPs。当时还没有成熟的 3D 并行框架，MFU 较低（估计远低于 40%）。

Llama 3 405B（Meta，2024 年 7 月）：训练在 30,720 张 H100 GPU 上，耗时约 54 天。使用了 Meta 内部深度定制的 FSDP + Megatron 混合方案，首次大规模采用 FP8 训练。MFU 达到约 38-43%。Meta 在 Infrastructure 博客中特别提到了两个挑战：16K+ GPU 集群的故障处理（平均 MTBF 调度）和训练中断后的快速恢复。Llama 3 的训练规模比 GPT-3 大了 3 倍的 GPU、2.5 倍的参数，但训练时间只增加了 3.6 倍——这说明框架的并行效率在大幅提升。

DeepSeek-V3 671B（DeepSeek，2024 年 12 月）：训练在 2,048 张 H800 上（H800 是 H100 的中文合规版本，NVLink 带宽被限制）。使用自研的 HAI-LLM 框架，主要在 DeepSpeed/Megatron 之上做定制。训练约 2 个月（含预训练 + 后训练）。核心创新是：完全 FP8 训练管线——这是第一次在大规模训练中全面验证 FP8 的可行性。MFU 达到约 50%+，在受限硬件（H800 限制的 NVLink 带宽）上反而实现了更高的硬件利用率。

这三个案例揭示了一个趋势：**同样规模的模型，2024 年比 2020 年的训练效率提升了 5-10 倍。** 一方面来自硬件（V100→H100，算力 10× 提升），另一方面来自框架——3D 并行 + FlashAttention + FP8 的组合优化。

3.7 硬件基础设施的演进

训练框架不能脱离硬件单独讨论。以下硬件层面的演进直接塑造了框架的设计方向：

NVIDIA DGX 系列的代际进化：

代际	发布	GPU	NVLink 带宽	对框架的意义
DGX-1	2016	8×P100	150 GB/s	TP 还不可行
DGX-2	2018	16×V100	300 GB/s	NVSwitch 引入
DGX A100	2020	8×A100	600 GB/s	TP 变得可行
DGX H100	2023	8×H100	900 GB/s	TP 成为标配
DGX B200	2025	8×B200	1.8 TB/s	FP8 训练成熟

其中 DGX A100 的 NVSwitch 架构是一个关键转折：它让节点内 8 张卡之间的通信不再需要经过 CPU 或网卡，而是通过 NVSwitch 全互联。这使得 TP 在 8 卡节点内的 AllReduce 通信延迟降低到可接受的范围。

网络拓扑的变化：

- InfiniBand：EDR（100Gb/s）→ HDR（200Gb/s）→ NDR（400Gb/s）→ XDR（800Gb/s, 2026）
- RoCE v2 在成本上有优势，但大规模场景下的 PFC 死锁和哈希冲突问题未完全解决
- 大厂训练集群普遍使用 2:1 收敛比的 Fat Tree 拓扑

HBM 容量 vs 模型规模的赛跑：

年份	代表性 GPU	HBM	代表性模型	参数
2018	V100 32GB	HBM2	BERT	340M
2020	A100 80GB	HBM2e	GPT-3	175B
2023	H100 80GB	HBM3	Llama 3	405B
2025	B200 141GB	HBM3e	—	~1T?

HBM 容量在 7 年里只增长了 4.4 倍，而同一时期的典型模型参数增长了 1,200 倍。这个剪刀差是 ZeRO 和 Offload 等技术存在的根本原因。

四、横向分析：2026 年训练框架竞争图谱

> 以 2026 年 4 月为时间切片。

4.1 主要玩家全景

框架	开发者	首次发布	GitHub Stars	开源	核心定位	关键区别
FSDP / PyTorch Distributed	Meta	2022-03	~99,541 (PyTorch)	✅	PyTorch 原生分片训练	生态集成最深
DeepSpeed	微软	2020-02	42,220	✅	大模型训练一站式方案	功能最全面
ColossalAI	潞晨科技	2022-08	41,372	✅	自动并行 + 异构内存	门槛最低
JAX	Google	2018-12	35,511	✅	TPU 训练 / 函数式	TPU 唯一路径
PyTorch Lightning	Lightning AI	2019	31,095	✅	训练流程抽象	开发体验最佳
NeMo	NVIDIA	2019	17,148	✅	端到端 AI 平台	最完整管线
Megatron-LM / Core	NVIDIA	2021-03	16,197	✅	超大规模训练	MFU 最高
Horovod	Uber	2017	14,691	✅	(维护模式)	已边缘化
OneFlow	一流科技	2020	9,391	✅	(开源停滞)	国产探索
LLM Foundry	Databricks	2023	~4,400	✅	MosaicML 生态	训练效率优化
FairScale	Meta	2020	3,408	✅	(已停止维护)	FSDP 前身

> Stars 数据来自 GitHub API 直接查询，2026-04-30 采集。

4.2 五个核心框架深度分析

DeepSpeed（微软）——功能最全的"瑞士军刀"

DeepSpeed 的核心优势不是某一个功能——是**功能的广度**。ZeRO 三阶段 + Offload + MoE + RLHF 优化 + 通信压缩 + 推理优化，几乎覆盖了训练全流程。

ZeRO 系列是 DeepSpeed 安身立命的根本。ZeRO-Offload 和 ZeRO-Infinity 是目前唯一成熟的显存卸载方案——FSDP 没有 Offload，ColossalAI 的 Gemini 在稳定性上还有差距。这意味着在显存受限的场景（单卡/少卡训练）下，DeepSpeed 几乎是唯一选择。

DeepSpeed-Chat（2023 年 3 月） 把 RLHF 训练从大厂的专利变成了可负担的开源方案。在 8×A100-80G 上 9 小时训练 13B 模型，成本 $290。这个"成本可计算"的能力对企业用户非常重要。

DeepSpeed-MoE（2024 年 3 月） 在 ZeRO 上增加了 Expert Parallelism，是生产环境中最早可用的 MoE 训练方案。

与 HuggingFace Transformers 的深度集成是 DeepSpeed 在中小企业市场的护城河。使用 Transformers + DeepSpeed 是最流行的"开箱即用"大模型训练方案。

劣势：

- 代码复杂度高，API 变化频繁——"ZeRO Stage 3 的配置参数有 50 多个"
- 本质上是"套在 PyTorch 外面的层"，跟 PyTorch 新特性的兼容性有时滞后
- 微软主导的开源项目，社区治理独立性存疑

Megatron-LM / Megatron Core（NVIDIA）——MFU 之王

Megatron 走的是一条跟 DeepSpeed 截然不同的路——**深度绑定 NVIDIA 硬件，追求极致利用率**。H100 上 FP8 训练的 MFU 达到 42-50%，是所有框架中最高的。

Tensor Parallelism 的原创者。 没有 Megatron 的 TP，3D 并行就不存在。今天的 NVSwitch + TP 组合是大规模训练不可或缺的基石。Megatron Core（2024 重构）把 TP 做成了模块化组件，支持更灵活的并行组合。

Pipeline Parallelism 的 1F1B 调度。 Megatron 在 PP 调度上做了大量工程优化，气泡（bubble）开销远小于 GPipe 的同步调度方案。

劣势：

- 生态闭环——只支持 NVIDIA GPU。AMD MI300X、华为昇腾、Google TPU 统统不支持
- 代码复杂度是所有框架中最高的——学习 Megatron 需要深入理解分布式系统
- 开源滞后策略到 2024 年的 Megatron Core 才真正改变

FSDP / PyTorch Distributed（Meta）——学起来最舒服的

FSDP 的设计哲学是"用最简单的方式做最核心的事"。它不做 Offload、不做 MoE EP、不做通信压缩，但把 ZeRO Stage 3 的功能跟 PyTorch 生态无缝融合。

跟 PyTorch 的原生集成是 FSDP 最强的护城河。FSDP 跟 `torch.compile`、DTensor、`torch.distributed.pipelining` 的协作是其他框架不能比的——DeepSpeed 的 ZeRO-3 跟 torch.compile 的兼容性至今不完美。

Llama 系列的大规模验证。 Meta 内部的 Llama 1/2/3 都是在 FSDP 定制版本上训练的。Llama 3 405B（30,720 H100）是 FSDP 的上限测试——证明了 FSDP 可以扩展到 3 万张 GPU 的规模。

劣势：

- 没有 Offload——ZeRO-Offload 和 ZeRO-Infinity 是 DeepSpeed 独有
- 没有 MoE EP——需要自己实现或切换到 DeepSpeed
- 超大规模深度优化不如 Megatron-DeepSpeed

存在一个有趣的矛盾：**FSDP 的"用户"最多**（PyTorch 的 99k Stars 所有用户理论上都可以用 FSDP），但**真正用 FSDP 大规模训练大模型的比例并不高**——大厂内部通常深度定制，中小团队更多用 HuggingFace + DeepSpeed。

从社区讨论来看，选择 FSDP 的主要场景是：① 不想引入 DeepSpeed 依赖的 PyTorch 项目；② 正在使用 torch.compile 等 PyTorch 新特性；③ 模型规模在 10B–70B 之间，不需要 Offload 和 MoE。选择 DeepSpeed 的场景是：① 模型超过 70B；② 需要 Offload（单卡训练或显存受限）；③需要 MoE 支持。

ColossalAI（潞晨科技）——自动化的探索者

ColossalAI 的差异化在于**降低门槛**。用户只需要加一行 `@colossalai.autoparallel`，框架自动在策略空间中搜索最优的并行组合。

ColossalAI Gemini 对标 ZeRO，支持 GPU + CPU + NVMe 三级存储的异构内存管理。在显存卸载的完整性上跟 ZeRO-Offload/Infinity 处于同一水平。

衍生项目： 从 ColossalAI 发展出 Open-Sora（视频生成训练）、Colossal-Inference（推理优化）。团队在多模态和推理方向上有技术布局。

现状： ~41k Stars 在开源社区有不小的影响力，但实际大规模训练部署远少于 DeepSpeed 和 Megatron。自动并行的理念很好，但"自动"在大规模场景下依然不如"专家手动调优"。

JAX + PaxML（Google）——TPU 的独有语言

JAX 不是为 PyTorch 用户设计的。它走了一条完全不同的路：**函数式编程 + XLA 编译 + TPU 原生支持。**

`jit`（即时编译）、`vmap`（向量化映射）、`pmap`（并行映射）等函数变换表达并行计算。PaxML 是 Google 在 JAX 上构建的大模型训练框架，PaLM 和 Gemini 系列都用的是 JAX + Pathways。

劣势极其明显： 跟 PyTorch 生态不兼容，学习曲线陡峭（要让熟悉 PyTorch 的工程师切换到 JAX 的函数式范式几乎不可能），TPU 绑定了训练平台（Google Cloud 用户才能用好）。

4.3 关键对比维度

维度	DeepSpeed	Megatron Core	FSDP	ColossalAI	JAX
并行策略	ZeRO+TP+PP+EP	TP+PP+SP+EP	ZeRO(FSDP)	自动并行	pmap+SPMD
FP8 训练	✅	✅（最佳）	❌	✅	✅（TPU）
MoE EP	✅	✅	❌	✅	✅
RLHF 优化	✅ 最佳	❌	❌	✅	❌
Offload 卸载	✅ 最佳	❌	❌	✅（Gemini）	❌
长上下文	Ulysses	自研 SP	社区方案	Ring Attn	自研
硬件绑定	通用 GPU	NVIDIA only	通用 GPU	通用 GPU	TPU 优先
易用性	中等	低	高	高	极低
MFU 典型值	35–45%	42–50%	30–40%	30–40%	40–50%(TPU)

4.4 用户群分布

- **Meta（Llama 系列）**：FSDP 定制。Llama 3 405B 在 30,720 H100 上训练 54 天，MFU ~38–43%
- **Microsoft（Phi、Copilot）**：DeepSpeed。自家框架深度优化
- **NVIDIA（Nemotron）**：Megatron Core。软硬一体验证平台
- **Google（PaLM、Gemini）**：JAX + Pathways。完全自研
- **OpenAI（GPT-4、o1）**：自研框架，与 Azure 深度绑定——具体未公开
- **DeepSeek**：自研 HAI-LLM（基于 PyTorch）。FP8 训练先行者，MFU ~50%+
- **Mistral、xAI**：Megatron-DeepSpeed 或 FSDP
- **学术/创业**：FSDP（首选，门槛低）或 ColossalAI（自动并行）

4.5 训练云平台

框架之上还有更上层的云平台。这层直接影响框架的选择——不同云平台对不同框架的优化深度不同。

NVIDIA DGX Cloud：预装 Megatron Core 的端到端训练服务。不只是"在云上跑 GPU"——NVIDIA 提供从集群规划、网络拓扑到框架调优的全套服务。硬件稀缺性是最大的壁垒：B200 集群不是谁都能排队买到的。使用 DGX Cloud 的用户往往选择 Megatron Core，因为 NVIDIA 在自家云上的优化最深。

AWS SageMaker：支持 DeepSpeed 和 FSDP，跟 S3/EFS 存储生态深度集成。优势是企业级合规和全球部署。SageMaker 的托管训练（Managed Training）提供了"上传代码 → 自动分配 GPU → 自动扩缩容"的体验，降低了训练的基础设施门槛。

Google Cloud TPU + GKE：JAX/Pathways 的唯一云平台。TPU v5p 和 Trillium（TPU v6）算力密度极高——单 Pod 超 8,000 芯片，但灵活性差：TPU 只适合训练，推理、微调、数据预处理都需要另外的方案。

Azure ND 系列：微软自家的 GPU 云，跟 DeepSpeed 做了最深的联合优化。GPT-4 和 Copilot 模型都在 Azure NDv5（H100）集群上训练。Azure 的网络（InfiniBand over Azure）在超大规模集群上的表现被广泛认为是三大云中最稳定的。

CoreWeave / Lambda Labs / RunPod：专业 AI 云，只做 GPU 算力。核心优势是"GPU 就是产品"——没有传统云厂商的虚拟机 overhead，价格通常比三大云便宜 30–50%。但缺乏 S3/GCS 级别的存储、网络、安全生态。

选择云平台本质上是在做一道权衡题： 部署深度（DGX Cloud 的 Megatron 优化最佳）vs 生态宽度（AWS 的集成最全）vs 成本（CoreWeave 最便宜）vs 合规（Azure 的企业安全最好）。框架的选择往往不是独立的——选了什么云，往往就决定了默认用什么框架。

五、横纵交汇洞察

5.1 历史如何塑造了今天的竞争格局

把纵向线和横向线叠在一起看，最突出的特征是：**今天的框架格局是 2019–2020 年"双星闪耀"的直接产物。** 几乎所有后续的发展——FSDP 的分叉、3D 并行的确立、ColossalAI 的入场——都可以追溯到 Megatron-LM 和 ZeRO 这两篇

论文。

DeepSpeed 和 Megatron-LM 在几乎同一时间提出了两种互补的方案——一个消除冗余（ZeRO），一个切分计算（TP）。两者的融合（Megatron-DeepSpeed，2021）至今是超大规模训练的技术基础。这个格局已经稳定了 5 年，且没有看到被打破的迹象。

FSDP 的诞生（2022 年 3 月）是唯一的重大格局变动。但不是因为技术——是因为**战略**。Meta 不想让 PyTorch 的分布式训练能力依赖微软。这个"分叉"虽然带来了两套 API 的社区分裂，但也催生了良性竞争：ZeRO-3 和 FSDP 都在追赶对方。

5.2 优势与劣势的历史根源

DeepSpeed 的核心优势——功能的广度——根植于 2020 年的激进扩张策略。 一年内连发三个 ZeRO 版本，从单卡覆盖到万卡。这种速度在后来的竞争中没有被复制。

Megatron Core 的极致 MFU——根植于 NVIDIA 的"软硬一体"路线。 其他框架等硬件出了再适配，Megatron 在新硬件还没上市时就做预研。但当用户不只用 NVIDIA 时，这个优势就变成了锁定的劣势——你无法在 AMD MI300X 上跑 Megatron Core 并期望同样的 MFU。

FSDP 的局限（没有 Offload）——根植于"简洁优先"的设计哲学。 Offload 涉及复杂的 CPU-GPU 调度和 PCIe 带宽管理，Meta 选择不做以保持 API 简洁。这限制了 FSDP 在显存受限场景下的可用性，但也让它更易用。

JAX 的局限（PyTorch 不兼容）——根植于 Google 在框架战争中的失败。 TensorFlow 在 2015-2022 年的框架战争中被 PyTorch 击败后，Google 选择了另辟蹊径。这不是技术选择，而是战略收缩——既然正面打不赢，就做一条独家赛道。

5.3 被抛弃的路线

回顾发展史，有几个方向在竞争中边缘化了：

- **纯 Pipeline Parallelism（GPipe 等）**：单用 PP 效率低（气泡问题），必须和 TP/DP 组合使用
- **Google Mesh TensorFlow / GShard**：跟 PyTorch 生态不兼容，影响力局限在 Google 内部
- **手动 Model Parallelism**：手工切分代码维护成本高，已让位于自动化的 TP/PP
- **纯 Data Parallelism（无 ZeRO）**：模型超过 1B 就不可用
- **单进程多线程 DataParallel**：GIL 瓶颈严重，PyTorch 已不再推荐

5.4 三个剧本

剧本一：生态整合，两强相争（60%）

DeepSpeed 和 Megatron Core 继续保持互补关系，FSDP 在中小规模场景扩大份额。2026-2027 年出现框架层标准的收敛——各家支持相同的并行策略组合，用户可以在 FSDP 和 DeepSpeed 之间透明切换。

这个剧本最可能的原因是大模型训练工具链已经非常成熟——再做一套新框架的边际收益很低。

剧本二：国产框架从"能用"到"好用"（25%）

华为昇腾 + 定制框架或 ColossalAI 在国产芯片上的适配，在出口管制加剧或国产芯片产能爆发时实现突破。国内大厂（字节、阿里、百度）加大在昇腾上的训练优化投入。

剧本三：范式切换颠覆现有框架（15%）

某种新的训练范式的出现——完全不需要传统反向传播的算法、神经拟态计算、或远超 Transformer 的新架构——让当前的并行策略积累失去价值。可能性不高，但回顾历史，AI 领域的"已知"往往只是阶段性认知。

5.5 国产框架的特殊处境

中国的大模型训练框架生态走了一条不同的路。DeepSeek 用的是自研 HAI-LLM，阿里通义千问用的是阿里巴巴 PAI 平台（基于 Megatron 深度定制），字节跳动的 veTrain 是火山引擎提供的托管训练服务，百度文心系列用 PaddlePaddle Fleet，华为昇腾用 MindSpore。

有几个值得注意的现象：

大厂几乎都自研或深度魔改，不用开源框架的"原版"。 DeepSeek 的 HAI-LLM 虽然基于 DeepSpeed/Megatron，但做了大量定制，特别是 FP8 训练管线。阿里 PAI 平台也在 Megatron 上做了深度修改以适配阿里云的硬件和网络拓扑。

华为 MindSpore 是国产框架中最特殊的一个。 它走的是"软硬一体"路线——从昇腾芯片到 MindSpore 框架到 MindStudio 开发工具，全方位自研。这种路线的上限很高（类似 NVIDIA + Megatron 的协同效应），但前提是昇腾芯片的性能和生态足够逼近 NVIDIA。2026 年的现状是：在昇腾 910B/910C 上跑训练，性能约为同级别 NVIDIA GPU 的 60-80%，兼容性问题仍然存在。

CoAI 是国产训练框架中最国际化的。 ColossalAI 虽然创始人在新加坡，但核心团队在中国。它在自动并行的差异化路线上做了不少学术贡献，但在工业落地时依然面临 DeepSpeed 的生态壁垒——用户问"为什么不用 DeepSpeed？"这个问题像咒语一样难破解。

5.6 一条贯穿始终的暗线

整个训练框架发展史本质上是一部**解耦与耦合的辩证关系史**：

ZeRO 把显存需求从"存全部"解耦为"存一份"，TP 又把多张卡的计算耦合在一起。FSDP 允许灵活配置哪些参数分片，Megatron Core 的 FP8 训练又把精度管理和计算管道紧密绑定。

更深层的是：**随着模型规模的指数级增长，我们越来越多地牺牲优雅性来换取可行性。** 一台 DGX H100 集群上运行训练任务涉及数万个有状态进程——GPU 线程、CPU 进程、NCCL 通信、存储客户端。框架的复杂度已经远超任何个人的理解范围。

系统越来越像一个"可控的黑箱"——我们理解它的配置接口，相信它能稳定工作，但已没有人能从代码层面完整理解整个训练栈的每一层。当你在万卡集群上训练万亿参数模型时，你只能测试出最稳定的配置组合然后信仰坚持，而不是理解每一行代码在做什么。

这可能是 AI 基础设施发展中最被低估的隐性成本：**我们正在用增长系统的不可理解性来换取算力的可扩展性。**

5.7 写在最后：训练框架会消失吗？

一个有趣的问题：五年后还有独立的"训练框架"这层抽象吗？

一种可能性是训练框架被硬件厂商深度集成——就像今天你不会说"我在用 CUDA 写程序"，而是说"我在 PyTorch 上训练"。类似地，未来可能你只需要说"我在 H100 集群上训练"，框架选择变成硬件厂商提供的默认配置。NVIDIA 的 Megatron Core 和 Google 的 Pathways 已经在朝这个方向走。

另一种可能性是训练框架被云平台吸收——你不再需要关心 DeepSpeed 还是 FSDP，云平台根据你的模型规模和硬件配置自动选择最优策略。AWS SageMaker 和 Google GKE 已经在做这件事。

但无论哪种方向，**框架解决的问题不会消失**——只是从"用户需要自己做选择"变成了"系统自动做选择"。分布式并行、显存管理、通信优化这些核心挑战永远存在。它们只是从用户可见的 API 变成了底层的系统逻辑。

六、信息来源

核心论文：

1. ZeRO — arXiv:1910.02054
2. Megatron-LM — arXiv:1909.08053
3. ZeRO-Offload — arXiv:2101.06840
4. ZeRO-Infinity — arXiv:2104.07857
5. Efficient Large-Scale Language Model Training Using Megatron-LM — arXiv:2104.04473
6. Colossal-AI — arXiv:2110.14883
7. Alpa: Automating Inter- and Intra-Operator Parallelism — arXiv:2201.12023 (OSDI 2022)
8. Reducing Activation Recomputation — arXiv:2205.05198
9. FlashAttention — arXiv:2205.14135 (NeurIPS 2022)
10. FlashAttention-2 — arXiv:2307.08691
11. FlashAttention-3 — arXiv:2407.08608
12. DeepSpeed Ulysses — arXiv:2309.14509
13. DeepSeek-V2 — arXiv:2405.04434
14. DeepSeek-V3 — arXiv:2412.19437
15. PaLM: Scaling Language Modeling with Pathways — arXiv:2204.02311

开源项目：

1. DeepSpeed — https://github.com/microsoft/DeepSpeed
2. Megatron-LM — https://github.com/NVIDIA/Megatron-LM
3. ColossalAI — https://github.com/hpcaitech/ColossalAI
4. PyTorch — https://github.com/pytorch/pytorch
5. JAX — https://github.com/google/jax

官方发布与博客：

1. DeepSpeed 开源 + ZeRO 发布 — Microsoft Research Blog (2020-02)
2. Megatron-DeepSpeed 合作 — Microsoft Research Blog (2021-05)
3. Megatron Core 发布 — NVIDIA Developer Blog (2024-07)

4. Llama 3 训练基础设施 — Meta AI Blog (2024-07)

数据来源：

1. GitHub Stars 数据 — GitHub API 直接查询 (2026-04-30)
2. DeepSpeed 论文作者信息 — arXiv + Microsoft Research
3. FlashAttention 作者信息 — NeurIPS 2022 + arXiv

> **方法论说明**：本报告采用横纵分析法（Horizontal-Vertical Analysis），由数字生命卡兹克（Khazix）提出，融合了语言学中的历时-共时分析（Saussure）、社会科学中的纵向-横截面研究设计、商学院案例研究法与竞争战略分析的核心思想。